

Distributed Key-Value Store

- Distributed Key-Value Store
 - Attribution
 - Summary
 - Quickstart - A Demonstration of Asynchronous Operations
 - Components
 - * Virtual Environment
 - * kvstore.proto
 - * `UpdatePublisher`
 - * `KeyValueService`
 - `WatchKey` RPC Implementation
 - * `KeyValueClient`
-

Attribution

Author

James Daniel Johnson

CWID

20183229

Course

CSIS 604 - Distributed Systems

Assignment

1.1 - Key-Value Store

Summary

This submission is an implementation of an in-memory key-value store. Key-value stores serve as backbones for many in-memory database systems in distributed systems within the computing industry, with prominent examples including Redis serving as caches, quick-access memory, etc.

The remote procedure calls offered by this system affords one the ability to accomplish standard CRUD operations such as `PUT`, `GET` or `DELETE`, as well as the ability to subscribe to updates using the `WatchKey` RPC.

Quickstart - A Demonstration of Asynchronous Operations

Start by ensuring that the virtual environment is activated

```
source .venv/bin/activate
```

Note: this step may be unnecessary if the `grpcio-tools` package has been previously installed globally.

With the requirements met, we can start the server and a watcher client in the background

```
python server.py & sleep 2 && python client.py watch foo &
[1] 96435
Server listening on port 50051
[2] 96448
INFO:UpdatePublisher:Subscribing listener id=139712979073712 to listen for updates to key=f
DEBUG:UpdatePublisher:Adding key=foo to subscriptions
```

This starts up the server and subscribes a background client to watch for updates to the key `foo`

We can run a simple GET operation with the client to see that `foo` does not currently have a value

```
$ python client.py get foo
No value associated with key=foo
```

Likewise, if we try to DELETE we'll receive an error message

```
$ python client.py delete foo
INFO:KeyValueService:Deleting entry with key foo
WARNING:KeyValueService:Could not find key foo in the store to delete
Key foo not found in the store.
```

However, we'll find that if we PUT an entry with `foo` as the key, we'll catch the update

```
$ python client.py put foo bar
INFO:KeyValueService:Putting entry {foo:bar}
DEBUG:KeyValueService:Publishing update to key foo
INFO:UpdatePublisher:Publishing update for key=foo
DEBUG:UpdatePublisher:Activating listener id=139712979073712 with update
DEBUG:KeyValueService:Received update event, appending to queue
DEBUG:KeyValueService:Found 1 updates in event queue, sending updates...
Successfully put foo:bar into the store!
Added entry 'foo' to the store.
Update: None -> foo:bar
```

The last line indicates that our watcher saw the entry updated from `None` to `{foo: bar}`.

If we conduct an idempotent PUT value again with the same value, no event will be published because the watcher watches for updates specifically.

```
$ python client.py put foo bar
INFO:KeyValueService:Putting entry {foo:bar}
Successfully put foo:bar into the store!
```

However, if we change the value, we'll get another update.

```
$ python client.py put foo baz
INFO:KeyValueService:Putting entry {foo:baz}
DEBUG:KeyValueService:Publishing update to key foo
INFO:UpdatePublisher:Publishing update for key=foo
DEBUG:UpdatePublisher:Activating listener id=139712979073712 with update
DEBUG:KeyValueService:Received update event, appending to queue
DEBUG:KeyValueService:Found 1 updates in event queue, sending updates...
Successfully put foo:baz into the store!
Updated entry 'foo'.
Update: foo:bar->foo:baz
```

Running a GET will simply return the value associated with the key.

```
$ python client.py get foo
"baz"
```

So repeated *idempotent* operations will not result in an update, but a successful DELETE operation will.

```
$ python client.py delete foo
INFO:KeyValueService:Deleting entry with key foo
DEBUG:KeyValueService:Found entry with key %s, deleting...
DEBUG:KeyValueService:Deletion of key foo successful, publishing update...
INFO:UpdatePublisher:Publishing update for key=foo
DEBUG:UpdatePublisher:Activating listener id=139712979073712 with update
DEBUG:KeyValueService:Received update event, appending to queue
DEBUG:KeyValueService:Found 1 updates in event queue, sending updates...
Successfully removed key foo from the store.
Deleted entry 'foo' from the store.
Update: foo:baz->None
```

If we kill the watcher thread, the service will unsubscribe the watcher.

```
$ kill %2
[2] + terminated python client.py watch foo
INFO:KeyValueService:Unsubscribing listener id=139712979073712 from watching key foo in Watcher
INFO:UpdatePublisher:Attempting to unsubscribe listener id=139712979073712 from watching key foo
DEBUG:UpdatePublisher:No more subscriptions for key=foo, removing key from watchlist
```

The service will also unsubscribe the watcher in the event of a timeout.

```
$ python client.py watch bah --timeout 1
INFO:UpdatePublisher:Subscribing listener id=139712979073536 to listen for updates to key=bah
```

```

DEBUG:UpdatePublisher:Adding key=bah to subscriptions
Watcher stopped because the timeout has been reached
INFO:KeyValueService:Unsubscribing listener id=139712979073536 from watching key bah in Wat
INFO:UpdatePublisher:Attempting to unsubscribe listener id=139712979073536 from watching key
DEBUG:UpdatePublisher:No more subscriptions for key=bah, removing key from watchlist

```

Finally, we can use a signal handler to stop the service by applying Ctrl+C.

```

$ jobs
[1] + running    python server.py
$ fg %1
[1] - running    python server.py
^C%

```

Components

Virtual Environment

A virtual environment has been created to support dependency management within this service implementation. Prior to running any python commands, be sure to activate the environment

```
source .venv/bin/activate
```

kvstore.proto

The `protobuf` specification outline the remote procedure calls (RPC) and message definitions provided as parameters and return values for each call. The specification for the services provided by this service is as specified.

```

service KeyValueStore {
  rpc PutKey (PutRequest) returns (PutResponse);
  rpc GetKey (GetRequest) returns (GetResponse);
  rpc DeleteKey (DeleteRequest) returns (DeleteResponse);
  rpc WatchKey (WatchRequest) returns (stream WatchResponse);
}

```

To generate the python files necessary to support the service, run the following command with the virtual environment activated in the terminal:

```
python -m grpc_tools.protoc -Iprotos --python_out=. --pyi_out=. --grpc_python_out=. protos/
```

UpdatePublisher

The mechanism used to publish updates consumed by the `WatchKey` RPC. Because the updates take place on a separate thread, a means of asynchronous

queuing of update events without contending for locks was desired. The publisher uses `threading.Lock` to secure its internal subscriptions internally:

```
class UpdatePublisher:
    """
    Publisher component used to publish
    event updates to listening threads
    """
    def __init__(self) -> None:
        self.subscriptions: dict[str, list[Callable[[UpdateEvent], None]]] = {}
        self._lock: Lock = Lock()
        self.logger: Logger = getLogger(self.__class__.__name__)

    def subscribe(self, key: str, listener: Callable[[UpdateEvent], None]):
        """
        Registers a listener with the publisher
        """
        self.logger.info(
            "Subscribing listener id=%d to listen for updates to key=%s",
            id(listener),
            key,
        )
        with self._lock:
            if key not in self.subscriptions:
                self.logger.debug("Adding key=%s to subscriptions", key)
                self.subscriptions[key] = []
            self.subscriptions[key].append(listener)

# ...
```

KeyValueService

The implementation of the `KeyValueService` interface. Manages the key-value store internally using a `threading.RLock`. The `RLock` lock type (or *re-entrant lock*) allows the thread that holds the lock to *re-enter* a critical section *so long as it already holds the lock*. This is useful for helper methods that may also require the use of the lock.

```
class KeyValueService(key_val_pb2_grpc.KeyValueStoreServicer):
    def __init__(self, shutdown_event: Event):
        self.store: dict = {}
        self.publisher: UpdatePublisher = UpdatePublisher()
        self._store_lock: RLock = RLock()
        self.shutdown_event: Event = shutdown_event
        self.logger: Logger = getLogger(self.__class__.__name__)

    def state(self, key) -> EntryState:
```

```

    """
    Returns the state of the store. Use of Reentrant Lock (`RLock`)
    allows the caller to have the key so long as it's on the same thread
    """
    with self._store_lock:
        exists = key in self.store
        return EntryState(
            exists=exists, key=key, value=self.store[key] if exists else None
        )

def PutKey(self, request, context):
    # Preamble with defaults
    key = request.key
    value = request.value
    is_update = False
    prev_state = None
    curr_state = None

    self.logger.info("Putting entry {%s:%s}", key, value)
    # Obtain key prior to mutating state
    with self._store_lock:
        prev_state = self.state(key)
        self.store[key] = value
        is_update = not prev_state.exists or prev_state.value != value
        curr_state = self.state(key)

    # Publish if update occurred
    if is_update:
        self.logger.debug("Publishing update to key %s", key)
        self.publisher.publish(
            key=key,
            update=UpdateEvent(
                old=prev_state,
                new=curr_state,
            ),
        )

    return key_val_pb2.PutResponse(
        message=f"Successfully put {key}:{value} into the store!", success=True
    )
# ...

```

Note that the service is responsible for determining whether an update actually occurred, and only publishes events when operations actually update the underlying values.

WatchKey RPC Implementation Each call to the WatchKey method uses a `threading.Condition` to establish when an event has been published.

```
#...
def WatchKey(self, request, context):
    key = request.key
    condition = Condition()
    event_queue = deque()

    def listener(update: UpdateEvent):
        self.logger.debug("Recieved update event, appending to queue")
        with condition:
            event_queue.append(update)
            condition.notify_all()

    self.publisher.subscribe(key=key, listener=listener)

    try:
        while context.is_active() and not self.shutdown_event.is_set():
            events = []
            with condition:
                while (
                    not event_queue
                    and context.is_active()
                    and not self.shutdown_event.is_set()
                ):
                    # Wait for up to one second
                    condition.wait(timeout=1.0)

                while event_queue:
                    events.append(event_queue.popleft())

            if len(events) > 0:
                self.logger.debug(
                    "Found %d updates in event queue, sending updates...",
                    len(events),
                )
            for update in events:
                yield WatchResponse(update=update)

    except grpc.RpcError as e:
        self.logger.error(
            "WatchKey stream terminated for key '%s'.\nCause: %s", key, e
        )
        context.set_code(grpc.StatusCode.CANCELLED)
        context.set_details(f"Stream interrupted.\nCause: {e!s}")
```

```

    finally:
        self.logger.info(
            "Unsubscribing listener id=%d from watching key %s in WatchKey",
            id(listener),
            key,
        )
        self.publisher.unsubscribe(key=key, listener=listener)

```

KeyValueClient

The client operates as a thin command-line client, and as such uses python's `argparse.ArgumentParser` to provide the user with commands, parameters, and a `--help` menu.

```

$ python client.py -h
usage: client.py [-h] {put,get,delete,watch} ...

```

Key-Value Store Client CLI

positional arguments:

{put,get,delete,watch}	Available commands
put	Upload a key-value pair
get	Get the value associated with a key
delete	Delete a key (and its associated value)
watch	Watch a given key for updates

options:

-h, --help	show this help message and exit
------------	---------------------------------

```

$ python client.py put --help
usage: client.py put [-h] key value

```

positional arguments:

key	The key to store
value	The value to store

options:

-h, --help	show this help message and exit
------------	---------------------------------

```

$ python client.py get -h
usage: client.py get [-h] key

```

positional arguments:

key	The key to search for
-----	-----------------------

```
options:
  -h, --help  show this help message and exit

$ python client.py delete -h
usage: client.py delete [-h] key

positional arguments:
  key                The key of the key-value pair to delete

options:
  -h, --help  show this help message and exit

$ python client.py watch -h
usage: client.py watch [-h] [--timeout TIMEOUT] key

positional arguments:
  key                The key to watch

options:
  -h, --help            show this help message and exit
  --timeout TIMEOUT    The timeout for the watcher in seconds
```